LangChain Agents & Agentic RAG

Complete Session Script

Duration: 20-25 minutes

Topics Covered:

What is LangChain

Building Agents with LangChain

Agent Implementation & Workflow

Agentic RAG vs Traditional RAG

Opening

Duration: 1 minute

Good morning/afternoon everyone. Today I'm going to walk you through something that's transforming how we build AI applications - LangChain agents and agentic RAG. By the end of our 25 minutes together, you'll understand what agents are, how to build them with LangChain, and how they're taking RAG systems from simple question-answering to intelligent, reasoning assistants.

Let me start with a quick scenario. Imagine you ask an AI: **What was our company's revenue last quarter and how does it compare to the industry average?** A traditional RAG system might retrieve a document and answer the first part, but completely miss that it needs to search the web for industry data. An agent, however, would reason through this, realize it needs multiple steps, fetch internal data, search externally, and then synthesize everything. That's the power we're exploring today.

Section 1: What is LangChain?

Duration: 3 minutes

So let's start with LangChain. LangChain is an open-source framework that makes it dramatically easier to build applications powered by large language models. Think of it as the scaffolding that connects LLMs to the real world.

Before LangChain, if you wanted to build an LLM application, you'd write a lot of boilerplate code - managing API calls, handling prompts, connecting to databases, parsing outputs. LangChain abstracts all of this into reusable components.

Core Concepts of LangChain

The framework is built around several core concepts:

First, there are chains - these are sequences of calls to LLMs or other utilities. For example, a simple chain might take user input, format it into a prompt, send it to an LLM, and parse the response.

Second, there are tools - these are functions or APIs that your LLM can use. Tools could be anything: a calculator, a web search API, a database query function, or your internal document retriever. LangChain standardizes how we describe these tools to the LLM.

Third, and most important for today, there are agents - these are systems where the LLM decides which tools to use and in what order to accomplish a task. The LLM becomes the reasoning engine that plans and executes.

LangChain also provides memory management for maintaining conversation context, document loaders for ingesting data, and integrations with dozens of LLM providers, vector databases, and external APIs.

Why LangChain Matters

The key insight is this: **LangChain transforms LLMs from simple text-in, text-out systems into reasoning engines that can interact with external tools and data sources.**

Instead of writing custom code for every integration, LangChain provides standardized interfaces. Instead of manually orchestrating complex workflows, you can let the LLM decide the flow. Instead of building everything from scratch, you can compose pre-built components.

Here's a simple example of what LangChain enables:

```
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

prompt = PromptTemplate(
    input_variables=['product'],
    template='Write a tagline for a product called {product}'
)

llm = ChatOpenAI(temperature=0.7)
chain = LLMChain(llm=llm, prompt=prompt)

result = chain.run(product='AI-powered coffee maker')
print(result)
```

This simple pattern - template, LLM, chain - is the foundation. But LangChain's real power emerges when we add agents that can reason and use tools autonomously.

Section 2: What is an Agent?

Duration: 4 minutes

Now let's dive into what an agent actually is. An agent is an LLM-powered system that can autonomously decide what actions to take to accomplish a goal. Unlike a fixed chain where we pre-define every step, an agent reasons about what to do next based on the current situation.

Chain vs Agent: The Core Difference

Think of the difference this way:

A **traditional chain** is like following a recipe exactly - step one, step two, step three. The path is predetermined. If step two fails or returns unexpected results, the chain doesn't adapt.

An **agent** is like a chef who looks at available ingredients and decides what to cook and how to prepare it. The agent evaluates the situation, makes decisions, and adapts based on what it discovers.

The Agent Loop: ReAct Pattern

Here's how an agent works, and this is crucial to understand. The process follows what's called the **ReAct pattern** - Reasoning and Acting. Let me walk you through the agent loop:

Agent Execution Cycle

Step 1: Receive Task

The agent receives a user query or task.

Step 2: Reason

The agent thinks: What do I need to do here? What information do I need? What tools are available to me? This reasoning is generated by the LLM itself.

Step 3: Decide Action

Based on reasoning, the agent decides to take an action. This might be calling a tool - searching a database, running a calculation, fetching a web page.

Step 4: Execute & Observe

The agent executes that action and observes the result. This observation becomes new information.

Step 5: Iterate or Conclude

The agent reasons again: Given this new information, am I done? Or do I need to do something else? This loop continues until the agent determines it has enough information, then generates a final response.

The beautiful part is that **the LLM is making these decisions at each step**. You're not hardcoding the logic. The agent adapts based on what it discovers.

Why Agents Are Powerful

Agents enable several powerful capabilities:

- **Multi-step reasoning:** Breaking complex tasks into subtasks automatically
- **Dynamic tool selection:** Choosing the right tool for each situation
- **Self-correction:** Recognizing when an action didn't work and trying alternatives
- **Adaptive workflows:** The execution path changes based on intermediate results
- **Complex problem solving:** Tackling problems that require multiple information sources

Traditional chains are deterministic and fast. Agents are flexible and intelligent, but slower and more expensive because of multiple LLM calls.

Section 3: Building Agents with LangChain

Duration: 6 minutes

Let me show you what this looks like in code and then we'll visualize the workflow with a concrete example.

Simple Agent Implementation

Here's a simple agent implementation using LangChain:

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import Tool
from langchain_openai import ChatOpenAI

def search_documents(query):
    return 'Q3 revenue was $50M'

def web_search(query):
    return 'Industry average Q3 revenue is $45M'

tools = [
    Tool(
        name='SearchDocs',
        func=search_documents,
        description='Searches internal company documents.'
    ),
    Tool(
        name='WebSearch',
        func=web_search,
        description='Searches the web for external information.'
    )
]

llm = ChatOpenAI(temperature=0)
agent = create_react_agent(llm, tools, prompt_template)

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True
)

result = agent_executor.invoke({
    'input': 'What was our Q3 revenue vs industry average?'
})
```

Agent Execution Walkthrough

Let's walk through what happens with the question: **What was our Q3 revenue and how does it compare to industry average?**

Real Execution Flow

Initial Reasoning

Thought: I need two pieces of information - our Q3 revenue and industry average. Let me start with internal data.

First Action

Action: SearchDocs
Action Input: Q3 revenue

First Observation

Observation: Q3 revenue was $50M

Second Reasoning

Thought: Good, I have our revenue. Now I need industry average. I'll use web search.

Second Action

Action: WebSearch
Action Input: industry average Q3 revenue

Second Observation

Observation: Industry average Q3 revenue is $45M

Final Reasoning

Thought: I now have both pieces. I can provide a complete answer.

Final Answer

> Our Q3 revenue was $50M, which is $5M above the industry average of $45M, representing 11% better performance.

This entire process - reasoning, tool selection, iteration - is **autonomous**. We didn't program this sequence.

Complete Working Example

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import Tool
from langchain_openai import ChatOpenAI

def calculator(expression):
    try:
        return str(eval(expression))
    except:
        return 'Error in calculation'

def get_temperature(city):
    temps = {'New York': '72F', 'London': '65F'}
    return temps.get(city, 'City not found')

tools = [
    Tool(
        name='Calculator',
        func=calculator,
        description='Math calculations. Input: expression.'
    ),
    Tool(
        name='WeatherAPI',
        func=get_temperature,
        description='Get temperature. Input: city name.'
    )
]

llm = ChatOpenAI(temperature=0, model='gpt-3.5-turbo')
agent = create_react_agent(llm, tools, prompt)

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,
    max_iterations=5
)

result = agent_executor.invoke({
    'input': 'What is the temperature in New York divided by 2?'
})

print(result['output'])
```

Section 4: Traditional RAG Recap

Duration: 2 minutes

Before we talk about agentic RAG, let me quickly recap traditional RAG.

How Traditional RAG Works

1. User asks a question
2. System converts question into embedding
3. Searches vector database
4. Retrieves top 3-5 chunks
5. Stuffs chunks into prompt
6. LLM generates answer

This works for straightforward questions like "What is our vacation policy?"

Where Traditional RAG Breaks Down

Poor retrieval: No way to try again differently.

Multi-source queries: Only retrieves from one source.

Multi-step reasoning: Can't handle complex reasoning.

Context management: No ability to summarize or expand.

Ambiguous queries: Treats all queries the same.

Traditional RAG is rigid and linear with no ability to adapt.

Section 5: Agentic RAG - The Evolution

Duration: 6 minutes

Agentic RAG solves the limitations of traditional RAG by making the RAG system itself an agent. Instead of a fixed retrieve-generate pipeline, we give the LLM agency over the entire retrieval process.

The Core Philosophy of Agentic RAG

Think about how you would research a complex topic. You don't just read the first book you find and stop there. You read, evaluate, realize you need more information, search for specific details, cross-reference sources, and iteratively build understanding. **Agentic RAG brings this human-like research process to AI systems.**

In traditional RAG, the system is passive - it retrieves once and hopes the results are good. In agentic RAG, the system is active - it's a researcher that can plan, execute, evaluate, and adapt its information-gathering strategy.

The fundamental shift is this: **retrieval becomes a decision, not a procedure.** The agent decides when to retrieve, what to retrieve, from where to retrieve, and whether the retrieved information is sufficient or if it needs to search again.

What Changes in Agentic RAG

Here's what fundamentally changes when we move from traditional to agentic RAG:

1. The Retriever Becomes a Tool

Instead of being hardwired into the pipeline, the retriever becomes one of many tools the agent can choose to use. The agent can use it multiple times, skip it entirely, or combine it with other tools like web search, calculators, or database queries.

2. Retrieval Becomes Iterative

The agent can retrieve, look at the results, decide they're insufficient, and retrieve again with a refined query. It's not limited to a single retrieval operation.

3. The Agent Reasons About Information Quality

After retrieving, the agent can evaluate: "Is this relevant? Is this enough? Do I have conflicting information that needs resolution?" Traditional RAG has no such evaluation step.

4. Query Reformulation

If initial retrieval fails, the agent can rephrase the query, break it into sub-queries, or try different search strategies. It adapts based on what it finds (or doesn't find).

The agent can reason about retrieval in ways traditional RAG cannot:

- Should I retrieve at all for this query?
- What exactly should I search for?
- Were the results good enough or too vague?
- Do I need to retrieve again with a different query?
- Should I retrieve from multiple different sources?
- Do I have enough information now to answer, or do I need more?

The Architecture of Agentic RAG

In agentic RAG, the architecture has several key components working together:

The Agent Core: An LLM (usually GPT-4 or Claude) that serves as the reasoning engine. It makes all decisions about what actions to take.

Retrieval Tools: One or more retrievers wrapped as tools. You might have different retrievers for different document types (policies vs technical docs) or different retrieval strategies (semantic vs keyword).

Complementary Tools: Web search for external data, calculators for computations, database queries for structured data, and any other capabilities the agent might need.

The Reasoning Loop: The ReAct pattern we discussed earlier, where the agent continuously reasons about what to do next based on what it has learned so far.

This architecture enables the agent to orchestrate complex information-gathering workflows that adapt to the specific question being asked.

Agentic RAG Example

Query: **Summarize key technical challenges in our Q3 reports and compare to competitors.**

Agentic RAG Execution

Reasoning 1

This is two-part. Need internal and competitor data from different sources.

Action 1

SearchInternalDocs: Q3 engineering challenges
Result: Scaling issues, infrastructure costs

Reasoning 2

Need competitor info from web.

Action 2

WebSearch: competitor technical challenges
Result: General industry articles (too broad)

Reasoning 3

Too general. Need specific competitors.

Action 3

SearchInternalDocs: main competitors
Result: Competitor names

Action 4

WebSearch: [specific competitors] challenges
Result: Relevant competitor data

Final Answer

Synthesizes comprehensive comparison

Key Agentic RAG Patterns

Pattern 1: Multi-step Retrieval - Retrieve, evaluate, retrieve again with refinements.

Pattern 2: Source Routing - Choose between different retrievers based on question type.

Pattern 3: Query Decomposition - Break complex questions into sub-questions.

Pattern 4: Self-Correction - Detect poor results and retry with different approach.

Pattern 5: Hybrid Search - Try semantic search, fallback to keyword, or combine both.

Implementation

```
from langchain.tools import Tool

def retrieve_docs(query):
    docs = retriever.get_relevant_documents(query)
    return '\n\n'.join([doc.page_content for doc in docs])

def web_search_function(query):
    return search_api.search(query)

retriever_tool = Tool(
    name='CompanyDocsRetriever',
    func=retrieve_docs,
    description='Retrieves from company documents.'
)

web_search_tool = Tool(
    name='WebSearch',
    func=web_search_function,
    description='Searches the internet.'
)

tools = [retriever_tool, web_search_tool]
agent = create_react_agent(llm, tools, prompt)
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    max_iterations=10
)
```

Comparison Table

Aspect	Traditional RAG	Agentic RAG
Workflow	Fixed	Adaptive
Retrieval	Single operation	Multiple iterations
Sources	One source	Multiple sources
Self-Correction	None	Can retry
Speed	Fast	Slower
Cost	Lower	Higher
Best For	Simple queries	Complex queries

Section 6: Benefits & Considerations

Duration: 2 minutes

Key Benefits

1. **Reliability** - Can adapt when initial retrieval fails.
2. **Complex Queries** - Handles multi-part questions requiring multiple sources.
3. **Better Accuracy** - Can verify, cross-reference, and recognize insufficient context.
4. **Transparency** - See the full reasoning process.

Trade-offs

Slower: Multiple LLM calls add latency.

More Expensive: Each reasoning step costs money.

Less Predictable: Agent chooses its own path.

Harder to Debug: Complex execution traces.

When to Use Each

Use Agentic RAG: Complex needs, accuracy over speed, multi-source requirements.

Use Traditional RAG: Simple Q&A, need speed, single source, predictable patterns.

Best Practice: Hybrid Approach

In production, use both. Route simple queries to traditional RAG and complex queries to agentic RAG. This optimizes for speed and capability.

Section 7: Implementation Best Practices

Duration: 1 minute

Tool Design Best Practices

1. Clear Tool Descriptions

State what the tool does, when to use it, and input format.

```
Good: 'Searches HR policies for benefits and leave info.
Input: natural language question'

Bad: 'Search documents'
```

2. Limit Tool Count

Start with 3-5 essential tools. Add more later.

3. Handle Errors Gracefully

Return informative error messages, not crashes.

4. Set Iteration Limits

Use max_iterations to prevent infinite loops.

5. Use Verbose Mode

Set verbose=True during development to see reasoning.

6. Monitor and Log

Log all interactions and reasoning steps for improvement.

Closing

Duration: 1 minute

Let me wrap up what we've covered.

LangChain is the framework that makes building intelligent systems practical. It provides chains, tools, and agents that connect LLMs to the real world.

Agents are LLM-powered systems that reason about actions using a loop of reasoning, acting, and observing. They make autonomous decisions based on tasks and available tools.

Agentic RAG transforms document retrieval into an intelligent process that can plan, adapt, and handle complexity.

> *The shift from traditional RAG to agentic RAG is like the difference between a vending machine and a concierge. One dispenses what you ask for. The other figures out what you need and goes to get it.*

By giving the LLM agency over the retrieval process, we unlock dramatically more powerful systems. We move from rigid pipelines to adaptive, intelligent workflows.

The trade-off is complexity, cost, and latency. But for problems that require it, agentic RAG delivers capabilities traditional approaches cannot match.

I'm happy to take questions about LangChain, agents, or implementing agentic RAG in our systems. Thank you.

Appendix: Delivery Tips

Pacing and Timing

- Speak at 130-150 words per minute
- Pause after code examples for 3-5 seconds
- Total runtime: 24-26 minutes
- If running short, expand workflow examples
- If running long, trim comparison table

Visual Aids

- Show agent loop diagram during Section 2
- Display code with syntax highlighting
- Use flowchart for agentic RAG execution
- Show side-by-side comparison of Traditional vs Agentic
- Have live demo ready (optional)

Audience Engagement

- Ask who has used ChatGPT or LLMs
- Ask who has built RAG systems
- Keep questions brief during session
- Save technical details for after
- Use relatable analogies

Common Questions Prepared

Q: Can agents make mistakes?

A: Yes. That's why we set max_iterations. Good tool descriptions help, but monitoring is essential.

Q: How much more expensive is agentic RAG?

A: Typically 3-10x more. Traditional might cost $0.01 per query, agentic $0.05-0.10.

Q: Can we use smaller models?

A: Yes, but GPT-4 reasons better than GPT-3.5. Test thoroughly.

Q: How to prevent wrong tool access?

A: Clear descriptions are critical. Add validation code if needed.

Q: Works with existing RAG?

A: Absolutely. Just wrap your retriever as a tool.

Quick Reference Guide

Building Your First Agent

Step 1: Install

pip install langchain langchain-openai

Step 2: Define Tools

Create functions for each capability

Step 3: Create Tool Objects

Wrap in Tool with names and descriptions

Step 4: Initialize Agent

Create with LLM, tools, and prompt

Step 5: Test and Iterate

Run queries, observe, refine

Minimal Agent (10 lines)

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import Tool
from langchain_openai import ChatOpenAI

tools = [Tool(name='Calc', func=lambda x: str(eval(x)),
              description='Math. Input: expression')]

agent = create_react_agent(ChatOpenAI(temperature=0), tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
result = executor.invoke({'input': 'What is 25 * 4?'})
```

RAG to Agentic RAG Checklist

1. Take existing retriever function
2. Wrap in Tool with description
3. Add other tools (web search, etc.)
4. Create agent with tools
5. Replace RAG chain with agent executor
6. Test with complex multi-step queries
7. Monitor and iterate on tool descriptions

Additional Resources

Documentation

- **LangChain Docs:** https://python.langchain.com/docs/
- **LangChain Agents Guide:** https://python.langchain.com/docs/modules/agents/
- **ReAct Paper:** Synergizing Reasoning and Acting in Language Models
- **LangChain GitHub:** https://github.com/langchain-ai/langchain

Tools and Frameworks

- **LangChain:** Primary framework for agents
- **LangSmith:** Monitoring and debugging
- **LlamaIndex:** Alternative with agent support
- **AutoGPT:** Autonomous agent framework

Advanced Topics

- Multi-agent systems and collaboration
- Memory systems for long-running agents
- Custom agent types beyond ReAct
- Agent evaluation and benchmarking
- Production deployment strategies
- Cost optimization techniques

Community

- **LangChain Discord:** Active community
- **GitHub Discussions:** Technical discussions
- **Twitter:** @langchainai for updates
- **YouTube:** LangChain tutorials

Duration: 20-25 minutes | Complete Implementation Guide

Full script with code examples, workflow diagrams, and best practices

For questions or feedback, contact your session coordinator